Tree maps

Sindre VC%gsdal

Tree maps

Library prep

install

activate

```
library(readxl)
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(treemapify)
library(viridis)
```

```
Loading required package: viridisLite
```

```r
library(tidyr)
library(tibble)
library(vegan)
```

```
Loading required package: permute
```

```r
library(knitr)
library(lubridate)
```

```
Attaching package: 'lubridate'

The following objects are masked from 'package:base':

    date, intersect, setdiff, union
```

```r
library(stringr)
library(forcats)
library(purrr)
library(patchwork)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v readr 2.1.5

-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

```r
library(broom)
```

Retrieve data

Home PC

```r
setwd("C:/Results")

Metabarcoding_results <- read_excel("Oslofjord_illumina_dada2.xlsx")

R_5_abundance_2023 <- read_excel(
  "C:/Program Files (x86)/Results/Planteplanktontellinger/R-5 2023/R-5_Ringdalsfjorden_kar
  na = "."
)
```

```
New names:
* `` -> `...1`
```

```r
R_5_abundance_2024 <- read_excel(
  "C:/Program Files (x86)/Results/Planteplanktontellinger/R-5 2024/R-5_Ringdalsfjorden_kar
  na = "."
)
```

```
New names:
* `` -> `...1`
```

```r
S_9_abundance_2023 <- read_excel(
  "C:/Program Files (x86)/Results/Planteplanktontellinger/S-9 2023/S-9_Haslau_karbon_2023.
 na = "."
)
```

```
New names:
* `` -> `...1`
```

```r
S_9_abundance_2024 <- read_excel(
  "C:/Program Files (x86)/Results/Planteplanktontellinger/S-9 2024/S-9_Haslau_karbon_2024.
  na = "."

)
```

```
New names:
* `` -> `...1`
```

data cleaning

metabarcoding

```
cols_to_remove <- c(
  "1-OS51", "2-OS52", "3-OS57", "4-OS23", "5-OS46", "6-OS30", "7-OS50",
  "8-OS17", "9-OS18", "10-OS19", "11-OS20", "12-OS21", "13-OS22", "14-OS24",
  "15-OS25", "16-OS26", "17-OS27", "18-OS28", "19-OS29", "20-OS31", "21-OS32",
  "22-OS41", "23-OS43", "24-OS44", "25-OS45", "26-OS47", "27-OS48", "28-OS49",
  "29-OS53", "30-OS54", "31-OS55", "32-OS56", "33-OS42", "34-OS44_2",
  "35-OS45_2", "36-OS47_2", "140-S28", "141-S29", "120-S8", "128-S16", "136-S24", "142-S30
)

Metabarcoding_results <- Metabarcoding_results %>%
  select(-any_of(cols_to_remove))

#removing all unrecorded ASV's

samples <- c("113-S1", "114-S2", "115-S3", "116-S4",
             "117-S5", "118-S6", "119-S7", "121-S9",
             "122-S10", "123-S11", "124-S12","125-S13","126-S14","127-S15",
             "129-S17","130-S18","131-S19","132-S20","133-S21","134-S22","135-S23",
             "137-S25", "138-S26","139-S27","143-S31")

# Option A: keep only rows where the sum across these columns > 0
metabarcoding_filtered <- Metabarcoding_results %>%
  filter(rowSums(across(all_of(samples)), na.rm = TRUE) > 0)

# removing "fungi" and "metazoa"
metabarcoding_filtered <- metabarcoding_filtered %>%
  mutate(subdivision_clean = tolower(trimws(subdivision))) %>%
  filter(! subdivision_clean %in% c("fungi", "metazoa", "perkinsea")) %>%
  select(-subdivision_clean)   # drop helper column if you don't need it

metabarcoding_filtered <- metabarcoding_filtered %>%
  mutate(subdivision = case_when(
    subdivision == "Gyrista" &
      class %in% c("Bacillariophyceae", "Mediophyceae", "Coscinodiscophyceae") ~ "Diatoms"
    TRUE ~ subdivision
  ))
```

microscopy

```
clean_abundance_data <- function(df) {
  df %>%
    # rename first two columns
    rename(higher_taxa = 1, Species = 2) %>%

    # remove rows where both columns are empty
    filter(!(is.na(higher_taxa) & is.na(Species))) %>%

    # fill down higher_taxa names
    fill(higher_taxa, .direction = "down") %>%

    # remove rows without species
    filter(!is.na(Species) & Species != "") %>%

    # remove summary rows
    filter(!(Species %in% c("Sum:", "Sum totalt:"))) %>%

    # convert "." to NA if any remain (for safety)
    mutate(across(everything(), ~na_if(., "."))) %>%

    # convert numeric columns to numeric if read as character
    mutate(across(-c(higher_taxa, Species), as.numeric)) %>%

    # convert abundance values to relative abundance (%)
    mutate(across(-c(higher_taxa, Species),
                  ~ (. / sum(., na.rm = TRUE)) * 100))
}

# Apply to your datasets
R_5_abundance_2023 <- clean_abundance_data(R_5_abundance_2023)
R_5_abundance_2024 <- clean_abundance_data(R_5_abundance_2024)
S_9_abundance_2023 <- clean_abundance_data(S_9_abundance_2023)
S_9_abundance_2024 <- clean_abundance_data(S_9_abundance_2024)




# target keywords and their canonical replacements
replacements <- c(
```

```
  chlorophyta      = "Chlorophyta",
  euglenophyceae   = "Euglenophyceae",
  cyanobacteria    = "Cyanobacteria",
  prasinophyceae   = "Prasinophyceae"
)

# combining datasets


# Example names:
df23 <- R_5_abundance_2023
df24 <- R_5_abundance_2024

keys <- c("higher_taxa", "Species")

# 1) Find column sets (excluding keys)
cols23 <- setdiff(names(df23), keys)
cols24 <- setdiff(names(df24), keys)

# Columns only in 2024 (these will be added to 2023 rows where keys match)
new_cols_from_24 <- setdiff(cols24, cols23)

# Columns present in both datasets
common_cols <- intersect(cols23, cols24)

# 2) Join to bring the 2024-only columns into df23 for matching rows
#    (left_join keeps all df23 rows; 2024-only columns will be NA for df23 rows with no ma
df23_augmented <- df23 %>%
  left_join(
    df24 %>% select(all_of(keys), all_of(new_cols_from_24)),
    by = keys
  )

# 3) Find rows that are in 2024 but not in 2023 (by keys) and prepare them to append
rows_only_24 <- anti_join(df24, df23, by = keys)


# add those missing columns to rows_only_24 with NA values.
cols_needed_in_24 <- setdiff(names(df23_augmented), names(rows_only_24))
if(length(cols_needed_in_24) > 0) {
  rows_only_24[cols_needed_in_24] <- NA
```

```r
}

cols_needed_in_23 <- setdiff(names(rows_only_24), names(df23_augmented))
if(length(cols_needed_in_23) > 0) {
  df23_augmented[cols_needed_in_23] <- NA
}

# 4) Bind rows: keep df23_augmented first (preserves original 2023 rows), then append 2024
combined_R_5 <- bind_rows(df23_augmented, rows_only_24) %>%
  # optional: reorder columns to put keys first
  select(all_of(keys), everything())

# Function to combine two data.frames with the described rules
combine_preserve_2023_then_append_2024 <- function(df23, df24, keys = c("higher_taxa", "Sp
  # Basic checks
  stopifnot(all(keys %in% names(df23)), all(keys %in% names(df24)))

  # Ensure keys in same order for joins (not strictly necessary but tidy)
  df23 <- df23 %>% select(all_of(keys), everything())
  df24 <- df24 %>% select(all_of(keys), everything())

  # Identify columns (excluding keys)
  cols23 <- setdiff(names(df23), keys)
  cols24 <- setdiff(names(df24), keys)

  # Columns only in 2024 that should be added to 2023 rows
  new_cols_from_24 <- setdiff(cols24, cols23)

  # 1) Add 2024-only columns into df23 for matching rows (left_join preserves df23 rows)
  df23_augmented <- df23 %>%
    left_join(df24 %>% select(all_of(keys), all_of(new_cols_from_24)),
              by = keys)

  # 2) Rows present only in 2024 (to append)
  rows_only_24 <- anti_join(df24, df23, by = keys)

  # 3) Ensure both data.frames have identical column sets and order before binding
  all_cols <- union(names(df23_augmented), names(rows_only_24))

  # Add missing columns as NA to each if needed
  for (cname in setdiff(all_cols, names(df23_augmented))) df23_augmented[[cname]] <- NA
```

```
  for (cname in setdiff(all_cols, names(rows_only_24))) rows_only_24[[cname]] <- NA

  # Reorder columns consistently
  df23_augmented <- df23_augmented %>% select(all_of(all_cols))
  rows_only_24 <- rows_only_24 %>% select(all_of(all_cols))

  # 4) Bind rows: df23 rows first, then 2024-only rows
  combined <- bind_rows(df23_augmented, rows_only_24)

  # Optional: keep original row order for df23 (already preserved), appended rows from df2
  # Return combined
  combined
}

# Apply to S_9 datasets
combined_S_9 <- combine_preserve_2023_then_append_2024(S_9_abundance_2023, S_9_abundance_2



combine_keep_both_date_columns <- function(primary_df, secondary_df,
                                           keys = c("higher_taxa", "Species"),
                                           secondary_suffix = "_S9",
                                           coerce_numeric = FALSE,
                                           date_cols = NULL) {
  stopifnot(all(keys %in% names(primary_df)), all(keys %in% names(secondary_df)))

  # Keep keys first
  primary_df <- primary_df %>% select(all_of(keys), everything())
  secondary_df <- secondary_df %>% select(all_of(keys), everything())

  # Identify non-key columns
  cols_primary <- setdiff(names(primary_df), keys)
  cols_secondary <- setdiff(names(secondary_df), keys)

  # Overlapping non-key columns
  overlapping <- intersect(cols_primary, cols_secondary)

  # If any overlap, rename those columns in the secondary by appending suffix.
  # Use rename_with to safely handle non-syntactic names.
  if (length(overlapping) > 0) {
    secondary_df <- secondary_df %>%
```

```
    rename_with(.cols = all_of(overlapping),
                .fn = ~ paste0(.x, secondary_suffix))
}

# Recompute secondary non-key columns after rename
cols_secondary_after <- setdiff(names(secondary_df), keys)

# Columns that will be added from secondary (includes renamed overlapping columns)
new_from_secondary <- setdiff(cols_secondary_after, cols_primary)

# Optional: coerce chosen date/read columns to numeric to ensure sums/plots work later
if (coerce_numeric) {
  if (is.null(date_cols)) {
    # coerce all non-key columns to numeric where possible
    cols_to_coerce <- union(cols_primary, cols_secondary_after)
  } else {
    cols_to_coerce <- date_cols
  }
  cols_to_coerce <- intersect(cols_to_coerce, names(primary_df))  # only existing cols
  if (length(cols_to_coerce) > 0) {
    primary_df <- primary_df %>% mutate(across(all_of(cols_to_coerce), ~ as.numeric(.)))
  }
  cols_to_coerce2 <- intersect(if (is.null(date_cols)) cols_secondary_after else date_co
  if (length(cols_to_coerce2) > 0) {
    secondary_df <- secondary_df %>% mutate(across(all_of(cols_to_coerce2), ~ as.numeric
  }
}

# 1) Left join primary with selected columns from secondary (keeps primary rows)
primary_aug <- primary_df %>%
  left_join(secondary_df %>% select(all_of(keys), all_of(new_from_secondary)), by = keys

# 2) Rows present only in secondary (renamed secondary used)
rows_only_secondary <- anti_join(secondary_df, primary_df, by = keys)

# 3) Ensure identical columns for binding
all_cols <- union(names(primary_aug), names(rows_only_secondary))
missing_in_primary <- setdiff(all_cols, names(primary_aug))
missing_in_secondary <- setdiff(all_cols, names(rows_only_secondary))

if (length(missing_in_primary) > 0) primary_aug[missing_in_primary] <- NA
```

```
  if (length(missing_in_secondary) > 0) rows_only_secondary[missing_in_secondary] <- NA

  # Reorder columns consistently and bind
  primary_aug <- primary_aug %>% select(all_of(all_cols))
  rows_only_secondary <- rows_only_secondary %>% select(all_of(all_cols))

  combined <- bind_rows(primary_aug, rows_only_secondary)
  combined
}

# Run with your objects (example):
combined_all <- combine_keep_both_date_columns(
  primary_df = combined_R_5,
  secondary_df = combined_S_9,
  keys = c("higher_taxa", "Species"),
  secondary_suffix = "_S9",
  coerce_numeric = FALSE
)

# Quick diagnostics
message("Dimensions: ", paste(dim(combined_all), collapse = " x "))
```

```
Dimensions: 293 x 32
```

```
message("Columns (first 20): ", paste(head(names(combined_all), 20), collapse = ", "))
```

```
Columns (first 20): higher_taxa, Species, 13/02/2023, 22/03/2023, 23/05/2023, 20/06/2023, 15/
```

```
combined_all <- combined_all %>%
  mutate(
    higher_taxa = as.character(higher_taxa),                    # ensure character
    higher_taxa = gsub("\\s*\\([^)]*\\)", "", higher_taxa),     # remove " ( ... )" includ
    higher_taxa = trimws(higher_taxa)                           # trim leading/trailing sp
  )

combined_all <- combined_all %>%
  mutate(higher_taxa = case_when(
    higher_taxa == "Bacillariophyceae" ~ "Diatoms",
    TRUE ~ higher_taxa
```

```
  ))
```

plots

metabarcoding

size based on number of ASV's

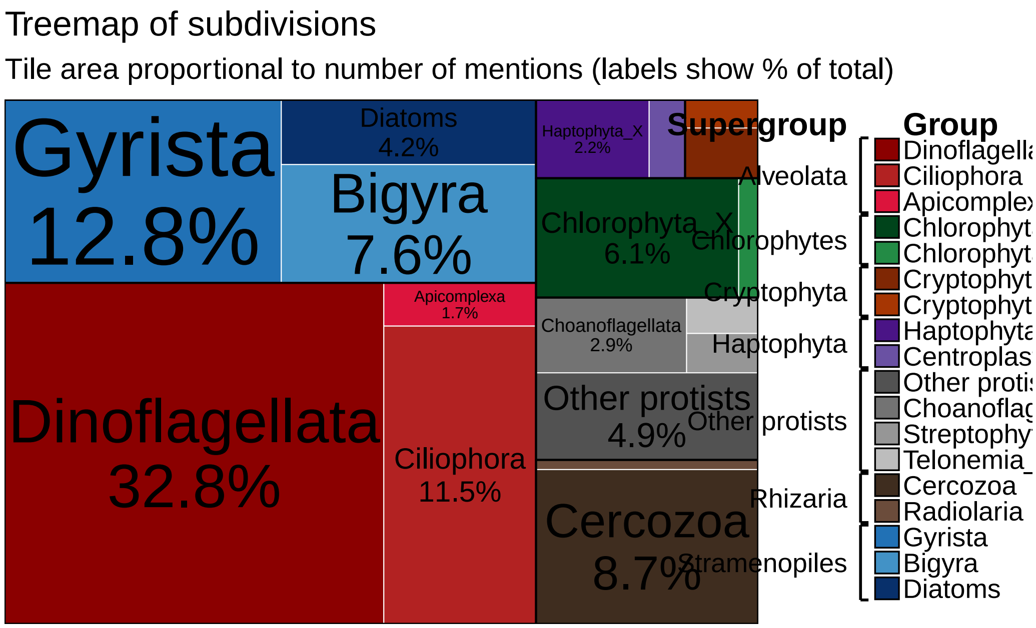
new tree maps

```
# ---------------------------
# DATA PREP (your workflow)
# ---------------------------
# Count subdivisions
sub_counts <- metabarcoding_filtered %>%
  filter(!is.na(subdivision), subdivision != "") %>%
  count(subdivision, name = "n") %>%
  arrange(desc(n))


sub_counts <- sub_counts %>%
  mutate(pct = n / sum(n) * 100) %>%
  mutate(subdivision = ifelse(pct < 0.5, "Other protists", subdivision)) %>%
  group_by(subdivision) %>%
  summarise(n = sum(n), .groups = "drop") %>%
  mutate(pct = n / sum(n) * 100)


sub_counts <- sub_counts %>%
  mutate(supergroup = case_when(
    subdivision %in% c("Dinoflagellata", "Dinophyceae", "Syndiniales", "Ciliophora", "Apic
    subdivision %in% c("Diatoms", "Gyrista", "Bigyra") ~ "Stramenopiles",
    subdivision %in% c("Chlorophyta", "Chlorophyta_X", "Chlorophyta_X:plas") ~ "Chlorophyt
    subdivision %in% c("Haptista", "Haptophyta_X", "Centroplasthelida_X") ~ "Haptophyta",
    subdivision %in% c("Cryptophyta_X:nucl", "Cryptophyta_X") ~ "Cryptophyta",
    subdivision %in% c("Cercozoa", "Radiolaria") ~ "Rhizaria",
    subdivision %in% c("Choanoflagellata", "Streptophyta_X", "Telonemia_X") ~ "Other proti
  TRUE ~ "Other protists"
```

```
  ))


palettes <- list(
  Alveolata = c("#8b0000", "#b22222", "#dc143c", "#ff4500", "#ff6a6a"),
  Stramenopiles = c("#2171b5","#4292c6" , "#08306b", "#6baed6", "#9ecae1"),
  Chlorophytes = c("#00441b", "#238b45", "#41ab5d", "#74c476", "#a1d99b"),
  Haptophyta = c("#4a1486", "#6a51a3", "#807dba", "#9e9ac8"),
  Cryptophyta = c("#7f2704", "#a63603", "#d94801", "#f16913"),
  Rhizaria = c("#3f2d1f", "#6b4c3b", "#8c6d5d", "#b08a6b"),
  `Other protists` = c("#525252", "#737373", "#969696", "#bdbdbd")
)


sub_counts <- sub_counts %>%
  group_by(supergroup) %>%
  mutate(supergroup_total = sum(n)) %>%
  ungroup()


sub_counts <- sub_counts %>%
  arrange(supergroup, desc(n)) %>%
  group_by(supergroup) %>%
  mutate(subdivision = factor(subdivision, levels = subdivision)) %>%
  ungroup()


sub_counts <- sub_counts %>%
  group_by(supergroup) %>%
  mutate(fill_col = palettes[[first(supergroup)]][seq_len(n())]) %>%
  ungroup()


# ---------------------------
# TREEMAP
# ---------------------------
treemap_plot <- ggplot(sub_counts,
       aes(area = n,
           fill = fill_col,
           subgroup = supergroup,
           label = paste0(subdivision, "\n", round(pct, 1), "%"))) +
```

```
  geom_treemap(color = "white") +
  geom_treemap_subgroup_border(color = "black", size = 1) +
  geom_treemap_text(colour = "black",
                    place = "centre",
                    grow = TRUE,
                    reflow = TRUE) +
  scale_fill_identity() +
  labs(
    title = "Treemap of subdivisions",
    subtitle = "Tile area proportional to number of mentions (labels show % of total)"
  ) +
  theme(legend.position = "none")


# ---------------------------
# LEGEND (hierarchical)
# ---------------------------
pal <- with(unique(sub_counts[, c("subdivision", "fill_col")]),
            setNames(fill_col, subdivision))

leg <- unique(sub_counts[, c("subdivision", "supergroup", "fill_col")])
leg$subdivision <- factor(leg$subdivision, levels = names(pal))
leg <- leg[order(leg$subdivision), ]
leg$y <- rev(seq_len(nrow(leg)))


br <- do.call(data.frame,
              aggregate(y ~ supergroup, leg,
                        function(x) c(lo = min(x), hi = max(x))))

br$ymin <- br$y.lo - 0.45
br$ymax <- br$y.hi + 0.45
br$ymid <- (br$ymin + br$ymax) / 2

br_multi <- subset(br, y.lo != y.hi)


p_leg <- ggplot() +

  geom_text(data = br,
            aes(x = 0, y = ymid, label = supergroup),
```

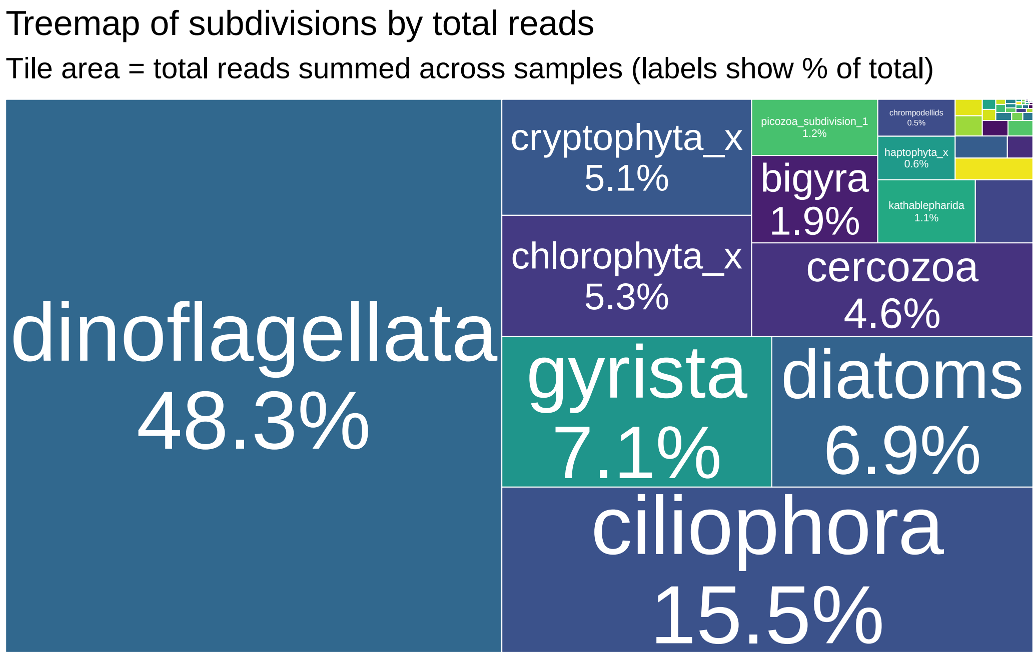
```
           hjust = 1, size = 3.4) +

  geom_segment(data = br_multi,
              aes(x = 0.25, xend = 0.25, y = ymin, yend = ymax)) +
  geom_segment(data = br_multi,
              aes(x = 0.25, xend = 0.4, y = ymin, yend = ymin)) +
  geom_segment(data = br_multi,
              aes(x = 0.25, xend = 0.4, y = ymax, yend = ymax)) +

  geom_tile(data = leg,
            aes(x = 0.75, y = y, fill = fill_col),
            width = 0.45, height = 0.85,
            color = "black", linewidth = 0.3) +

  geom_text(data = leg,
            aes(x = 1.05, y = y, label = subdivision),
            hjust = 0, size = 3.4) +

  # Change these two labels:
  annotate("text", x = 0, y = max(leg$y) + 1,
           label = "Supergroup",
           hjust = 1, fontface = "bold", size = 4.2) +

  annotate("text", x = 1.05, y = max(leg$y) + 1,
           label = "Group",
           hjust = 0, fontface = "bold", size = 4.2) +

  scale_fill_identity() +
  coord_cartesian(xlim = c(-1.2, 3), clip = "off") +
  theme_void()

# ---------------------------
# FINAL COMBINED PLOT
# ---------------------------
final_plot <- treemap_plot + p_leg +
  plot_layout(widths = c(4, 1.3))

print(final_plot)
```



size based on number of reads

```
# Define the sample/read columns
samples <- c("113-S1", "114-S2", "115-S3", "116-S4",
             "117-S5", "118-S6", "119-S7", "121-S9",
             "122-S10", "123-S11", "124-S12","125-S13","126-S14","127-S15",
             "129-S17","130-S18","131-S19","132-S20","133-S21","134-S22","135-S23",
             "137-S25", "138-S26","139-S27","143-S31")

# Compute reads per row and aggregate by subdivision
sub_reads <- metabarcoding_filtered %>%
  mutate(subdivision = ifelse(is.na(subdivision), NA_character_,
                              trimws(tolower(subdivision)))) %>%
  filter(!is.na(subdivision), subdivision != "") %>%
  # ensure sample columns are numeric; convert if necessary
  mutate(across(all_of(samples), ~ as.numeric(.))) %>%
  rowwise() %>%
  mutate(reads_sum = sum(c_across(all_of(samples)), na.rm = TRUE)) %>%
  ungroup() %>%
  group_by(subdivision) %>%
  summarise(total_reads = sum(reads_sum, na.rm = TRUE)) %>%
```

```
  arrange(desc(total_reads))

# Optional: group small subdivisions into "other"
threshold_reads <- 1000   # set threshold as desired
sub_reads2 <- sub_reads %>%
  mutate(subdivision2 = ifelse(total_reads <= threshold_reads, "other", subdivision)) %>%
  group_by(subdivision2) %>%
  summarise(total_reads = sum(total_reads)) %>%
  arrange(desc(total_reads))

sub_reads <- sub_reads %>%
  mutate(pct = total_reads / sum(total_reads) * 100)

# Option A: percent only in label
ggplot(sub_reads, aes(area = total_reads,
                      fill = subdivision,
                      label = paste0(subdivision, "\n", sprintf("%.1f%%", pct)))) +
  geom_treemap(color = "white") +
  geom_treemap_text(colour = "white", place = "centre", grow = TRUE, reflow = TRUE, min.si
  scale_fill_viridis_d(option = "D") +
  labs(title = "Treemap of subdivisions by total reads",
       subtitle = "Tile area = total reads summed across samples (labels show % of total)"
       fill = "Subdivision") +
  theme(legend.position = "none")
```



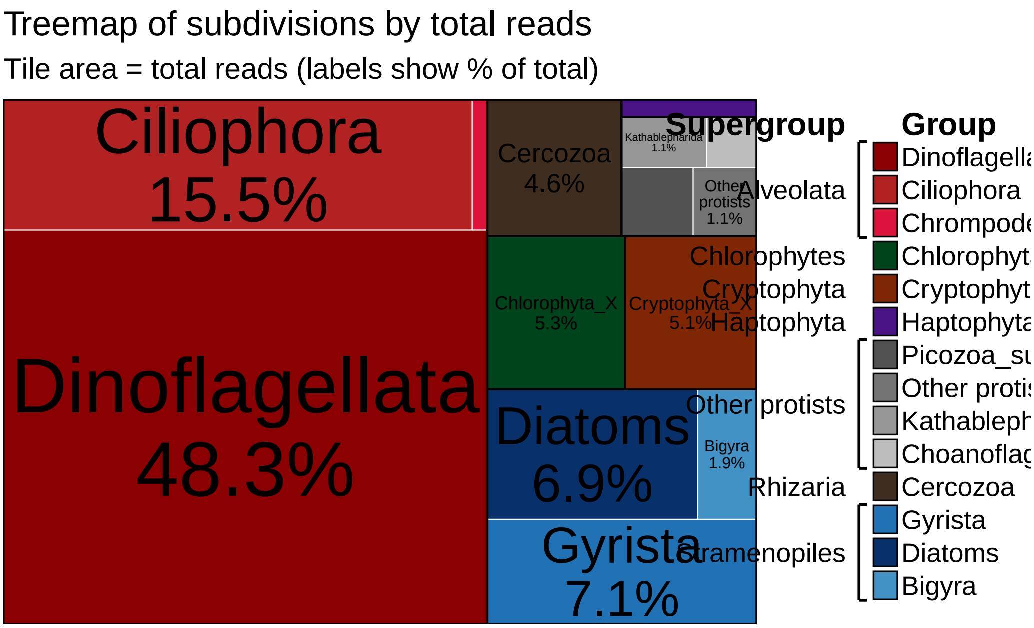
new tree maps

```
sub_reads <- sub_reads %>%
  mutate(subdivision = str_to_title(subdivision))

sub_reads <- sub_reads %>%
  mutate(
    subdivision = subdivision %>%
      str_to_title() %>%
      str_replace_all("_x", "_X")
)

sub_reads <- sub_reads %>%
  mutate(pct = total_reads / sum(total_reads) * 100) %>%
  mutate(subdivision = ifelse(pct < 0.5, "Other protists", subdivision)) %>%
  group_by(subdivision) %>%
  summarise(total_reads = sum(total_reads), .groups = "drop") %>%
  mutate(pct = total_reads / sum(total_reads) * 100)


sub_reads <- sub_reads %>%
```

```r
  mutate(supergroup = case_when(
    subdivision %in% c("Dinoflagellata", "Dinophyceae", "Syndiniales", "Ciliophora", "Apic
    subdivision %in% c("Gyrista", "Bigyra", "Diatoms") ~ "Stramenopiles",
    subdivision %in% c("Chlorophyta", "Chlorophyta_X", "Chlorophyta_X:plas") ~ "Chlorophyt
    subdivision %in% c("Haptista", "Haptophyta_X", "Centroplasthelida_X") ~ "Haptophyta",
    subdivision %in% c("Cryptophyta_X:nucl", "Cryptophyta_X") ~ "Cryptophyta",
    subdivision %in% c("Cercozoa", "Radiolaria") ~ "Rhizaria",
    subdivision %in% c("Choanoflagellata", "Streptophyta_X", "Telonemia_X") ~ "Other proti
    TRUE ~ "Other protists"
  ))

sub_reads <- sub_reads %>%
  group_by(supergroup) %>%
  mutate(
    supergroup_total = sum(total_reads),
    supergroup_pct = supergroup_total / sum(total_reads)
  ) %>%
  ungroup()

sub_reads <- sub_reads %>%
  arrange(supergroup, desc(total_reads)) %>%
  group_by(supergroup) %>%
  mutate(subdivision = factor(subdivision, levels = subdivision)) %>%
  ungroup()

palettes <- list(
  Alveolata = c("#8b0000", "#b22222", "#dc143c", "#ff4500", "#ff6a6a"),
  Stramenopiles = c("#2171b5","#08306b" ,"#4292c6" , "#6baed6", "#9ecae1"),
  Chlorophytes = c("#00441b", "#238b45", "#41ab5d", "#74c476", "#a1d99b"),
  Haptophyta = c("#4a1486", "#6a51a3", "#807dba", "#9e9ac8"),
  Cryptophyta = c("#7f2704", "#a63603", "#d94801", "#f16913"),
  Rhizaria = c("#3f2d1f", "#6b4c3b", "#8c6d5d", "#b08a6b"),
  `Other protists` = c("#525252", "#737373", "#969696", "#bdbdbd")
)

sub_reads <- sub_reads %>%
  group_by(supergroup) %>%
  mutate(
    fill_col = palettes[[supergroup[1]]][seq_len(n())]
  ) %>%
  ungroup()
```

```r
p_tm <- ggplot(sub_reads,
       aes(area = total_reads,
           fill = fill_col,
           subgroup = supergroup,
           label = paste0(subdivision, "\n", sprintf("%.1f%%", pct)))) +
  geom_treemap(color = "white") +
  geom_treemap_subgroup_border(color = "black", size = 1) +
  geom_treemap_text(colour = "black", place = "centre",
                    grow = TRUE, reflow = TRUE, min.size = 3) +
  scale_fill_identity() +
  labs(title = "Treemap of subdivisions by total reads",
       subtitle = "Tile area = total reads (labels show % of total)") +
  theme(legend.position = "none")

pal <- with(unique(sub_reads[, c("subdivision", "fill_col")]),
            setNames(fill_col, subdivision))



leg <- unique(sub_reads[, c("subdivision", "supergroup", "fill_col")])
leg$subdivision <- factor(leg$subdivision, levels = names(pal))
leg <- leg[order(leg$subdivision), ]
leg$y <- rev(seq_len(nrow(leg)))

br <- do.call(data.frame, aggregate(y ~ supergroup, leg,
        function(x) c(lo = min(x), hi = max(x))))
br$ymin <- br$y.lo - 0.45
br$ymax <- br$y.hi + 0.45
br$ymid <- (br$ymin + br$ymax) / 2
br_multi <- subset(br, y.lo != y.hi)

p_leg <- ggplot() +

  geom_text(data = br,
            aes(x = 0, y = ymid, label = supergroup),
            hjust = 1, size = 3.4) +

  geom_segment(data = br_multi,
               aes(x = 0.25, xend = 0.25, y = ymin, yend = ymax)) +
```

```r
  geom_segment(data = br_multi,
               aes(x = 0.25, xend = 0.4, y = ymin, yend = ymin)) +
  geom_segment(data = br_multi,
               aes(x = 0.25, xend = 0.4, y = ymax, yend = ymax)) +

  geom_tile(data = leg,
            aes(x = 0.75, y = y, fill = fill_col),
            width = 0.45, height = 0.85,
            color = "black", linewidth = 0.3) +

  geom_text(data = leg,
            aes(x = 1.05, y = y, label = subdivision),
            hjust = 0, size = 3.4) +

  # Change these two labels:
  annotate("text", x = 0, y = max(leg$y) + 1,
           label = "Supergroup",
           hjust = 1, fontface = "bold", size = 4.2) +

  annotate("text", x = 1.05, y = max(leg$y) + 1,
           label = "Group",
           hjust = 0, fontface = "bold", size = 4.2) +

  scale_fill_identity() +
  coord_cartesian(xlim = c(-1.2, 3), clip = "off") +
  theme_void()


p_tm + p_leg + plot_layout(widths = c(4, 1.3))
```

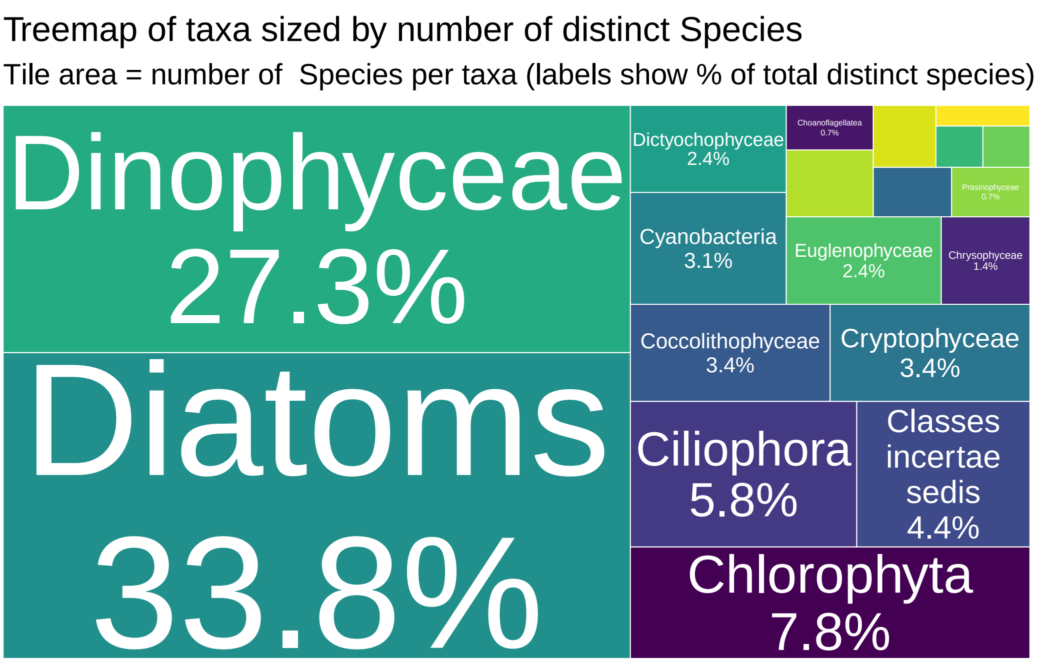


microscopy

size based on number of taxa

```
# Prepare counts: distinct Species per higher_taxa
counts_ht <- combined_all %>%
  mutate(
    higher_taxa = ifelse(is.na(higher_taxa), NA_character_, trimws(as.character(higher_tax
    Species = ifelse(is.na(Species), NA_character_, trimws(as.character(Species)))
  ) %>%
  filter(!is.na(higher_taxa), higher_taxa != "") %>%
  # normalize species strings to avoid duplicates from capitalization/whitespace if desire
  mutate(Species = tolower(Species)) %>%
  group_by(higher_taxa) %>%
  summarise(
    n_species = n_distinct(Species, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(n_species)) %>%
  mutate(pct = n_species / sum(n_species) * 100)
```

```r
# Treemap: label shows higher_taxa and percent of total distinct species
ggplot(counts_ht, aes(area = n_species,
                      fill = higher_taxa,
                      label = paste0(higher_taxa, "\n", sprintf("%.1f%%", pct)))) +
  geom_treemap(color = "white") +
  geom_treemap_text(colour = "white", place = "centre", grow = TRUE, reflow = TRUE, min.si
  scale_fill_viridis_d(option = "D") +
  labs(
    title = "Treemap of taxa sized by number of distinct Species",
    subtitle = "Tile area = number of  Species per taxa (labels show % of total distinct s
    fill = "higher_taxa"
  ) +
  theme(legend.position = "none")
```



new plots

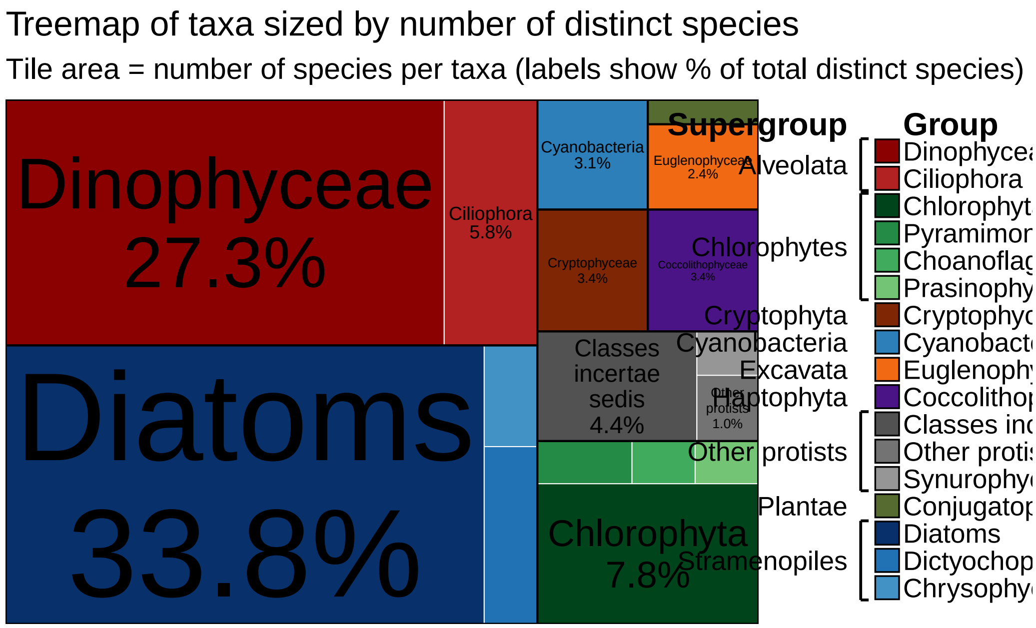
```r
# ---------------------------
# 1. Collapse small groups
# ---------------------------
counts_ht <- counts_ht %>%
```

```r
  mutate(pct = n_species / sum(n_species) * 100) %>%
  mutate(higher_taxa = ifelse(pct < 0.5, "Other protists", higher_taxa)) %>%
  group_by(higher_taxa) %>%
  summarise(n_species = sum(n_species), .groups = "drop") %>%
  mutate(pct = n_species / sum(n_species) * 100)


# --------------------------
# 2. Supergroup mapping
# --------------------------
counts_ht <- counts_ht %>%
  mutate(supergroup = case_when(
    higher_taxa %in% c("Dinophyceae","Ciliophora") ~ "Alveolata",
    higher_taxa %in% c("Diatoms", "Dictyochophyceae","Chrysophyceae") ~ "Stramenopiles",
    higher_taxa %in% c("Chlorophyta", "Pyramimonadophyceae", "Choanoflagellatea", "Prasino
    higher_taxa %in% c("Conjugatophyceae") ~ "Plantae",
    higher_taxa %in% c("Coccolithophyceae") ~ "Haptophyta",
    higher_taxa %in% c("Cryptophyceae") ~ "Cryptophyta",
    higher_taxa %in% c("Cyanobacteria") ~ "Cyanobacteria",
    higher_taxa %in% c("Euglenophyceae") ~ "Excavata",
    higher_taxa %in% c("Classes incertae sedis", "Other protists", "Synurophyceae") ~ "Oth
    TRUE ~ "Other protists"
  ))


# --------------------------
# 3. Palettes (consistent with your earlier scheme)
# --------------------------
palettes <- list(
  Alveolata = c("#8b0000", "#b22222", "#dc143c", "#ff4500", "#ff6a6a"),
  Stramenopiles = c("#08306b", "#2171b5", "#4292c6", "#6baed6", "#9ecae1"),
  Chlorophytes = c("#00441b", "#238b45", "#41ab5d", "#74c476", "#a1d99b"),
Plantae = c("#556b2f", "#6b8e23", "#808000", "#9acd32", "#bdb76b"),
Haptophyta = c("#4a1486", "#6a51a3", "#807dba", "#9e9ac8"),
  Cryptophyta = c("#7f2704", "#a63603", "#d94801", "#f16913"),
  Cyanobacteria = c("#2c7fb8", "#41b6c4", "#7fcdbb"),
Excavata = c("#f16913", "#006d2c", "#2ca25f", "#66c2a4", "#99d8c9"),
`Other protists` = c("#525252", "#737373", "#969696", "#bdbdbd")
)
```

```r
# ---------------------------
# 4. Ordering
# ---------------------------
counts_ht <- counts_ht %>%
  arrange(supergroup, desc(n_species)) %>%
  group_by(supergroup) %>%
  mutate(higher_taxa = factor(higher_taxa, levels = higher_taxa)) %>%
  ungroup()


# ---------------------------
# 5. Assign colors
# ---------------------------
counts_ht <- counts_ht %>%
  group_by(supergroup) %>%
  mutate(fill_col = palettes[[first(supergroup)]][seq_len(n())]) %>%
  ungroup()


# ---------------------------
# 6. TREEMAP
# ---------------------------
p_tm <- ggplot(counts_ht,
       aes(area = n_species,
           fill = fill_col,
           subgroup = supergroup,
           label = paste0(higher_taxa, "\n", sprintf("%.1f%%", pct)))) +
  geom_treemap(color = "white") +
  geom_treemap_subgroup_border(color = "black", size = 1) +
  geom_treemap_text(colour = "black",
                    place = "centre",
                    grow = TRUE,
                    reflow = TRUE,
                    min.size = 3) +
  scale_fill_identity() +
  labs(
    title = "Treemap of taxa sized by number of distinct species",
    subtitle = "Tile area = number of species per taxa (labels show % of total distinct sp
  ) +
  theme(legend.position = "none")
```

```r
# ---------------------------
# 7. LEGEND (same system as your working plot)
# ---------------------------
pal <- with(unique(counts_ht[, c("higher_taxa", "fill_col")]),
            setNames(fill_col, higher_taxa))

leg <- unique(counts_ht[, c("higher_taxa", "supergroup", "fill_col")])
leg$higher_taxa <- factor(leg$higher_taxa, levels = names(pal))
leg <- leg[order(leg$higher_taxa), ]
leg$y <- rev(seq_len(nrow(leg)))


br <- do.call(data.frame,
              aggregate(y ~ supergroup, leg,
                        function(x) c(lo = min(x), hi = max(x))))

br$ymin <- br$y.lo - 0.45
br$ymax <- br$y.hi + 0.45
br$ymid <- (br$ymin + br$ymax) / 2

br_multi <- subset(br, y.lo != y.hi)


p_leg <- ggplot() +

  geom_text(data = br,
            aes(x = 0, y = ymid, label = supergroup),
            hjust = 1, size = 3.4) +

  geom_segment(data = br_multi,
               aes(x = 0.25, xend = 0.25, y = ymin, yend = ymax)) +
  geom_segment(data = br_multi,
               aes(x = 0.25, xend = 0.4, y = ymin, yend = ymin)) +
  geom_segment(data = br_multi,
               aes(x = 0.25, xend = 0.4, y = ymax, yend = ymax)) +

  geom_tile(data = leg,
            aes(x = 0.75, y = y, fill = fill_col),
            width = 0.45, height = 0.85,
            color = "black", linewidth = 0.3) +
```

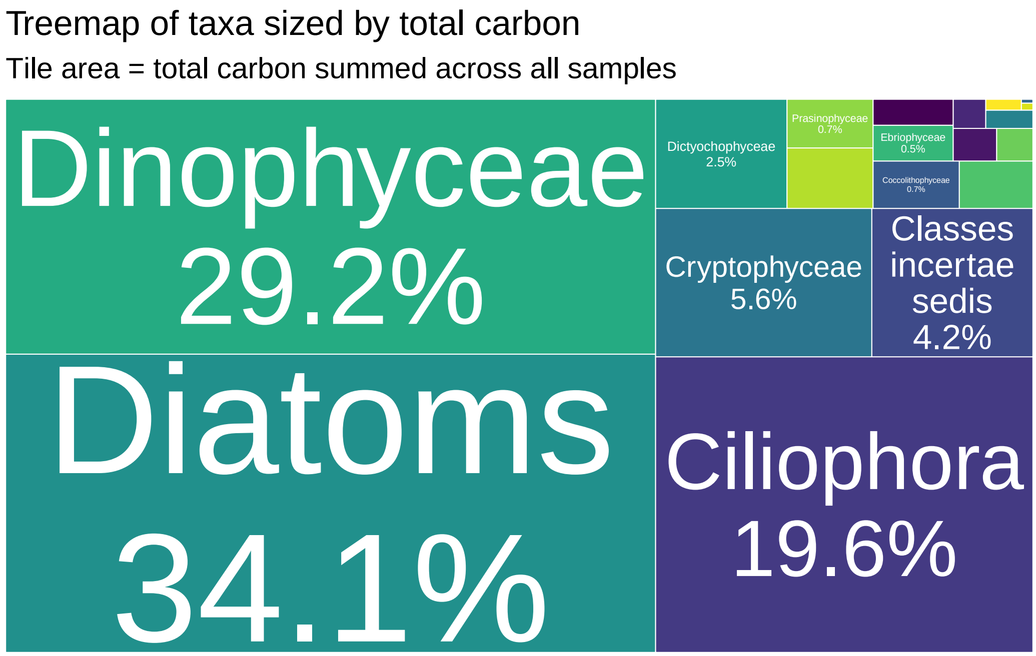
```r
  geom_text(data = leg,
            aes(x = 1.05, y = y, label = higher_taxa),
            hjust = 0, size = 3.4) +

  annotate("text", x = 0, y = max(leg$y) + 1,
           label = "Supergroup",
           hjust = 1, fontface = "bold", size = 4.2) +

  annotate("text", x = 1.05, y = max(leg$y) + 1,
           label = "Group",
           hjust = 0, fontface = "bold", size = 4.2) +

  scale_fill_identity() +
  coord_cartesian(xlim = c(-1.2, 3), clip = "off") +
  theme_void()


# ---------------------------
# 8. FINAL COMBINED PLOT
# ---------------------------
final_plot <- p_tm + p_leg +
  plot_layout(widths = c(4, 1.3))

print(final_plot)
```



size based on amount of carbon

```
# Keys
keys <- c("higher_taxa", "Species")

# 1) Identify non-key columns (assumed to be date/read/carbon columns)
nonkey_cols <- setdiff(names(combined_all), keys)

# 2) Coerce non-key cols to numeric where possible, compute row-wise carbon sum
#    (treat NA as 0)
combined_carbon <- combined_all %>%
  # keep keys as character and trim
  mutate(
    higher_taxa = ifelse(is.na(higher_taxa), NA_character_, trimws(as.character(higher_tax
    Species = ifelse(is.na(Species), NA_character_, trimws(as.character(Species)))
  ) %>%
  # coerce all non-key columns to numeric (suppress warnings for non-convertible values)
  mutate(across(all_of(nonkey_cols), ~ suppressWarnings(as.numeric(.)))) %>%
  # replace NA in numeric columns with 0 for summing
  mutate(across(all_of(nonkey_cols), ~ replace_na(.x, 0))) %>%
  # compute total carbon per row (sum across the non-key numeric columns)
  rowwise() %>%
```

```r
  mutate(total_carbon_row = sum(c_across(all_of(nonkey_cols)), na.rm = TRUE)) %>%
  ungroup()

# 3) Aggregate total carbon by higher_taxa
carbon_by_ht <- combined_carbon %>%
  filter(!is.na(higher_taxa), higher_taxa != "") %>%
  group_by(higher_taxa) %>%
  summarise(total_carbon = sum(total_carbon_row, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(total_carbon)) %>%
  mutate(pct = total_carbon / sum(total_carbon) * 100)

# 4) Treemap: area by total_carbon, label shows higher_taxa and percent of total carbon
ggplot(carbon_by_ht, aes(area = total_carbon,
                         fill = higher_taxa,
                         label = paste0(higher_taxa, "\n", sprintf("%.1f%%", pct)))) +
  geom_treemap(color = "white") +
  geom_treemap_text(colour = "white", place = "centre", grow = TRUE, reflow = TRUE, min.si
  scale_fill_viridis_d(option = "D") +
  labs(
    title = "Treemap of taxa sized by total carbon",
    subtitle = "Tile area = total carbon summed across all samples",
    fill = "higher_taxa"
  ) +
  theme(legend.position = "none")
```



new plot

```
# ---------------------------
# 1. Collapse small taxa (<0.5%)
# ---------------------------
carbon_by_ht <- carbon_by_ht %>%
  mutate(pct = total_carbon / sum(total_carbon) * 100) %>%
  mutate(higher_taxa = ifelse(pct < 0.5, "Other protists", higher_taxa)) %>%
  group_by(higher_taxa) %>%
  summarise(total_carbon = sum(total_carbon), .groups = "drop") %>%
  mutate(pct = total_carbon / sum(total_carbon) * 100)


# ---------------------------
# 2. Supergroup mapping
# (same structure as your species plot)
# ---------------------------
carbon_by_ht <- carbon_by_ht %>%
  mutate(supergroup = case_when(
    higher_taxa %in% c("Dinophyceae","Ciliophora") ~ "Alveolata",
    higher_taxa %in% c("Diatoms", "Dictyochophyceae","Chrysophyceae") ~ "Stramenopiles",
```

```
    higher_taxa %in% c("Chlorophyta", "Pyramimonadophyceae", "Choanoflagellatea", "Prasino
    higher_taxa %in% c("Conjugatophyceae") ~ "Plantae",
    higher_taxa %in% c("Coccolithophyceae") ~ "Haptophyta",
    higher_taxa %in% c("Cryptophyceae") ~ "Cryptophyta",
    higher_taxa %in% c("Cyanobacteria") ~ "Cyanobacteria",
    higher_taxa %in% c("Euglenophyceae") ~ "Excavata",
    higher_taxa %in% c("Classes incertae sedis", "Other protists", "Synurophyceae") ~ "Oth
    TRUE ~ "Other protists"
  ))


# --------------------------
# 3. Palettes (same system as previous plots)
# --------------------------
palettes <- list(
  Alveolata = c("#8b0000", "#b22222", "#dc143c", "#ff4500", "#ff6a6a"),
  Stramenopiles = c("#08306b", "#2171b5", "#4292c6", "#6baed6", "#9ecae1"),
  Chlorophytes = c("#00441b", "#238b45", "#41ab5d", "#74c476", "#a1d99b"),
Plantae = c("#556b2f", "#6b8e23", "#808000", "#9acd32", "#bdb76b"),
Haptophyta = c("#4a1486", "#6a51a3", "#807dba", "#9e9ac8"),
  Cryptophyta = c("#7f2704", "#a63603", "#d94801", "#f16913"),
  Cyanobacteria = c("#2c7fb8", "#41b6c4", "#7fcdbb"),
Excavata = c("#f16913", "#66c2a4", "#99d8c9"),
`Other protists` = c("#525252", "#737373", "#969696", "#bdbdbd")
)


# --------------------------
# 4. Ordering
# --------------------------
carbon_by_ht <- carbon_by_ht %>%
  arrange(supergroup, desc(total_carbon)) %>%
  group_by(supergroup) %>%
  mutate(higher_taxa = factor(higher_taxa, levels = higher_taxa)) %>%
  ungroup()


# --------------------------
# 5. Assign colors
# --------------------------
carbon_by_ht <- carbon_by_ht %>%
```

```
  group_by(supergroup) %>%
  mutate(fill_col = palettes[[first(supergroup)]][seq_len(n())]) %>%
  ungroup()


# ---------------------------
# 6. TREEMAP
# ---------------------------
p_tm <- ggplot(carbon_by_ht,
       aes(area = total_carbon,
           fill = fill_col,
           subgroup = supergroup,
           label = paste0(higher_taxa, "\n", sprintf("%.1f%%", pct)))) +
  geom_treemap(color = "white") +
  geom_treemap_subgroup_border(color = "black", size = 1) +
  geom_treemap_text(colour = "black",
                    place = "centre",
                    grow = TRUE,
                    reflow = TRUE,
                    min.size = 3) +
  scale_fill_identity() +
  labs(
    title = "Treemap of taxa sized by total carbon",
    subtitle = "Tile area = total carbon summed across all samples"
  ) +
  theme(legend.position = "none")


# ---------------------------
# 7. LEGEND (same hierarchical system)
# ---------------------------
pal <- with(unique(carbon_by_ht[, c("higher_taxa", "fill_col")]),
            setNames(fill_col, higher_taxa))

leg <- unique(carbon_by_ht[, c("higher_taxa", "supergroup", "fill_col")])
leg$higher_taxa <- factor(leg$higher_taxa, levels = names(pal))
leg <- leg[order(leg$higher_taxa), ]
leg$y <- rev(seq_len(nrow(leg)))


br <- do.call(data.frame,
```

```r
               aggregate(y ~ supergroup, leg,
                         function(x) c(lo = min(x), hi = max(x))))

br$ymin <- br$y.lo - 0.45
br$ymax <- br$y.hi + 0.45
br$ymid <- (br$ymin + br$ymax) / 2

br_multi <- subset(br, y.lo != y.hi)


p_leg <- ggplot() +

  geom_text(data = br,
            aes(x = 0, y = ymid, label = supergroup),
            hjust = 1, size = 3.4) +

  geom_segment(data = br_multi,
               aes(x = 0.25, xend = 0.25, y = ymin, yend = ymax)) +
  geom_segment(data = br_multi,
               aes(x = 0.25, xend = 0.4, y = ymin, yend = ymin)) +
  geom_segment(data = br_multi,
               aes(x = 0.25, xend = 0.4, y = ymax, yend = ymax)) +

  geom_tile(data = leg,
            aes(x = 0.75, y = y, fill = fill_col),
            width = 0.45, height = 0.85,
            color = "black", linewidth = 0.3) +

  geom_text(data = leg,
            aes(x = 1.05, y = y, label = higher_taxa),
            hjust = 0, size = 3.4) +

  annotate("text", x = 0, y = max(leg$y) + 1,
           label = "Supergroup",
           hjust = 1, fontface = "bold", size = 4.2) +

  annotate("text", x = 1.05, y = max(leg$y) + 1,
           label = "Group",
           hjust = 0, fontface = "bold", size = 4.2) +

  scale_fill_identity() +
```

```
  coord_cartesian(xlim = c(-1.2, 3), clip = "off") +
  theme_void()


# ---------------------------
# 8. FINAL OUTPUT
# ---------------------------
final_plot <- p_tm + p_leg +
  plot_layout(widths = c(4, 1.3))

print(final_plot)
```